```
import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models

dataset_choice = 'mnist'       # 'mnist'
epochs = 50
batch_size = 128
noise_dim = 100
learning_rate = 0.0002
save_interval = 5


print("GPU Available:", tf.config.list_physical_devices('GPU'))


os.makedirs("generated_samples", exist_ok=True)
os.makedirs("final_generated_images", exist_ok=True)


if dataset_choice == 'mnist':
    (x_train, y_train), _ = tf.keras.datasets.mnist.load_data()
else:
    (x_train, y_train), _ = tf.keras.datasets.fashion_mnist.load_data()

x_train = (x_train.astype("float32") - 127.5) / 127.5
x_train = np.expand_dims(x_train, axis=-1)

BUFFER_SIZE = x_train.shape[0]
dataset = (
    tf.data.Dataset.from_tensor_slices(x_train)
    .shuffle(BUFFER_SIZE)
    .batch(batch_size, drop_remainder=True)
    .prefetch(tf.data.AUTOTUNE)
)

img_shape = (28, 28, 1)


def build_generator():
    return models.Sequential([
        layers.Dense(256, input_shape=(noise_dim,)),
        layers.LeakyReLU(0.2),
        layers.BatchNormalization(),

        layers.Dense(512),
        layers.LeakyReLU(0.2),
        layers.BatchNormalization(),

        layers.Dense(np.prod(img_shape), activation='tanh'),
        layers.Reshape(img_shape)
    ])


def build_discriminator():
    return models.Sequential([
        layers.Flatten(input_shape=img_shape),
        layers.Dense(512),
        layers.LeakyReLU(0.2),

        layers.Dense(256),
        layers.LeakyReLU(0.2),

        layers.Dense(1, activation='sigmoid')
    ])

generator = build_generator()
discriminator = build_discriminator()


cross_entropy = tf.keras.losses.BinaryCrossentropy()

g_optimizer = tf.keras.optimizers.Adam(learning_rate, 0.5)
d_optimizer = tf.keras.optimizers.Adam(learning_rate, 0.5)

@tf.function
def train_step(real_images):
    batch = tf.shape(real_images)[0]
    noise = tf.random.normal([batch, noise_dim])
```

```
    with tf.GradientTape() as disc_tape:
        fake_images = generator(noise, training=True)

        real_output = discriminator(real_images, training=True)
        fake_output = discriminator(fake_images, training=True)

        d_loss_real = cross_entropy(tf.ones_like(real_output), real_output)
        d_loss_fake = cross_entropy(tf.zeros_like(fake_output), fake_output)
        d_loss = d_loss_real + d_loss_fake

    grads_d = disc_tape.gradient(d_loss, discriminator.trainable_variables)
    d_optimizer.apply_gradients(zip(grads_d, discriminator.trainable_variables))

    noise = tf.random.normal([batch, noise_dim])

    with tf.GradientTape() as gen_tape:
        fake_images = generator(noise, training=True)
        fake_output = discriminator(fake_images, training=True)
        g_loss = cross_entropy(tf.ones_like(fake_output), fake_output)

    grads_g = gen_tape.gradient(g_loss, generator.trainable_variables)
    g_optimizer.apply_gradients(zip(grads_g, generator.trainable_variables))

    return d_loss, g_loss

def save_image_grid(epoch, n=5):
    noise = tf.random.normal([n*n, noise_dim])
    gen_imgs = generator(noise, training=False)
    gen_imgs = (gen_imgs + 1) / 2.0

    fig, axs = plt.subplots(n, n, figsize=(5, 5))
    idx = 0
    for i in range(n):
        for j in range(n):
            axs[i, j].imshow(gen_imgs[idx, :, :, 0], cmap='gray')
            axs[i, j].axis('off')
            idx += 1
    fig.savefig(f"generated_samples/epoch_{epoch:02d}.png")
    plt.close()

d_losses = []
g_losses = []


for epoch in range(1, epochs + 1):
    start = tf.timestamp()

    epoch_d_loss = []
    epoch_g_loss = []

    for real_batch in dataset:
        d_loss, g_loss = train_step(real_batch)

        epoch_d_loss.append(d_loss.numpy())
        epoch_g_loss.append(g_loss.numpy())

    # Store average loss per epoch
    d_losses.append(np.mean(epoch_d_loss))
    g_losses.append(np.mean(epoch_g_loss))

    elapsed = tf.timestamp() - start

    print(
        f"Epoch {epoch}/{epochs} | "
        f"D_loss: {d_losses[-1]:.3f} | "
        f"G_loss: {g_losses[-1]:.3f} | "
        f"Time: {elapsed:.2f}s"
    )

    if epoch % save_interval == 0:
        save_image_grid(epoch)



noise = tf.random.normal([100, noise_dim])
final_imgs = generator(noise, training=False)
final_imgs = (final_imgs + 1) / 2.0

for i in range(100):
    plt.imshow(final_imgs[i, :, :, 0], cmap='gray')
    plt.axis('off')
```

```
    plt.savefig(f"final_generated_images/img_{i}.png")
    plt.close()


classifier = models.Sequential([
    layers.Flatten(input_shape=img_shape),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

classifier.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

classifier.fit(x_train, y_train, epochs=5, batch_size=128, verbose=0)

predictions = classifier.predict(final_imgs)
predicted_labels = np.argmax(predictions, axis=1)

print("\nLabel Distribution of Generated Images:")
unique, counts = np.unique(predicted_labels, return_counts=True)
for u, c in zip(unique, counts):
    print(f"Class {u}: {c} images")
```

```
Epoch 8/50 | D_loss: 1.161 | G_loss: 0.926 | Time: 2.52s
Epoch 9/50 | D_loss: 1.163 | G_loss: 0.930 | Time: 2.44s
Epoch 10/50 | D_loss: 1.164 | G_loss: 0.930 | Time: 3.09s
Epoch 11/50 | D_loss: 1.163 | G_loss: 0.931 | Time: 2.48s
Epoch 12/50 | D_loss: 1.164 | G_loss: 0.939 | Time: 2.47s
Epoch 13/50 | D_loss: 1.162 | G_loss: 0.944 | Time: 2.48s
Epoch 14/50 | D_loss: 1.163 | G_loss: 0.947 | Time: 2.64s
Epoch 15/50 | D_loss: 1.155 | G_loss: 0.955 | Time: 3.01s
Epoch 16/50 | D_loss: 1.152 | G_loss: 0.964 | Time: 2.47s
Epoch 17/50 | D_loss: 1.151 | G_loss: 0.968 | Time: 2.44s
Epoch 18/50 | D_loss: 1.148 | G_loss: 0.974 | Time: 2.45s
Epoch 19/50 | D_loss: 1.146 | G_loss: 0.983 | Time: 3.12s
Epoch 20/50 | D_loss: 1.146 | G_loss: 0.987 | Time: 2.47s
Epoch 21/50 | D_loss: 1.141 | G_loss: 0.996 | Time: 2.46s
Epoch 22/50 | D_loss: 1.139 | G_loss: 0.999 | Time: 2.47s
Epoch 23/50 | D_loss: 1.139 | G_loss: 1.003 | Time: 2.57s
Epoch 24/50 | D_loss: 1.138 | G_loss: 1.010 | Time: 3.01s
Epoch 25/50 | D_loss: 1.137 | G_loss: 1.014 | Time: 2.46s
Epoch 26/50 | D_loss: 1.133 | G_loss: 1.020 | Time: 2.48s
Epoch 27/50 | D_loss: 1.129 | G_loss: 1.020 | Time: 2.53s
Epoch 28/50 | D_loss: 1.127 | G_loss: 1.032 | Time: 3.09s
Epoch 29/50 | D_loss: 1.126 | G_loss: 1.038 | Time: 2.49s
Epoch 30/50 | D_loss: 1.126 | G_loss: 1.042 | Time: 2.49s
Epoch 31/50 | D_loss: 1.119 | G_loss: 1.051 | Time: 2.47s
Epoch 32/50 | D_loss: 1.118 | G_loss: 1.055 | Time: 3.67s
Epoch 33/50 | D_loss: 1.116 | G_loss: 1.061 | Time: 2.66s
Epoch 34/50 | D_loss: 1.113 | G_loss: 1.065 | Time: 2.47s
Epoch 35/50 | D_loss: 1.114 | G_loss: 1.074 | Time: 2.49s
Epoch 36/50 | D_loss: 1.112 | G_loss: 1.079 | Time: 2.72s
Epoch 37/50 | D_loss: 1.108 | G_loss: 1.080 | Time: 2.89s
Epoch 38/50 | D_loss: 1.104 | G_loss: 1.085 | Time: 2.53s
Epoch 39/50 | D_loss: 1.105 | G_loss: 1.091 | Time: 2.48s
Epoch 40/50 | D_loss: 1.107 | G_loss: 1.091 | Time: 2.47s
Epoch 41/50 | D_loss: 1.101 | G_loss: 1.100 | Time: 3.68s
Epoch 42/50 | D_loss: 1.100 | G_loss: 1.107 | Time: 2.47s
Epoch 43/50 | D_loss: 1.098 | G_loss: 1.112 | Time: 2.47s
Epoch 44/50 | D_loss: 1.096 | G_loss: 1.115 | Time: 2.44s
Epoch 45/50 | D_loss: 1.094 | G_loss: 1.120 | Time: 2.70s
Epoch 46/50 | D_loss: 1.092 | G_loss: 1.122 | Time: 2.93s
Epoch 47/50 | D_loss: 1.089 | G_loss: 1.129 | Time: 3.76s
Epoch 48/50 | D_loss: 1.088 | G_loss: 1.131 | Time: 2.50s
Epoch 49/50 | D_loss: 1.086 | G_loss: 1.139 | Time: 5.10s
Epoch 50/50 | D_loss: 1.084 | G_loss: 1.147 | Time: 2.54s
1/4 ━━━━━━━━━━━━━━━━━━━━ 0s 174ms/stepWARNING:tensorflow:5 out of the last 3612 calls to <function TensorFlowTrainer.make_
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 58ms/step

Label Distribution of Generated Images:
Class 0: 9 images
Class 1: 15 images
Class 2: 7 images
Class 3: 15 images
Class 4: 5 images
Class 5: 6 images
Class 6: 8 images
Class 7: 16 images
Class 8: 9 images
Class 9: 10 images
```

```
plt.figure(figsize=(8, 5))
plt.plot(d_losses, label="Discriminator Loss")
```

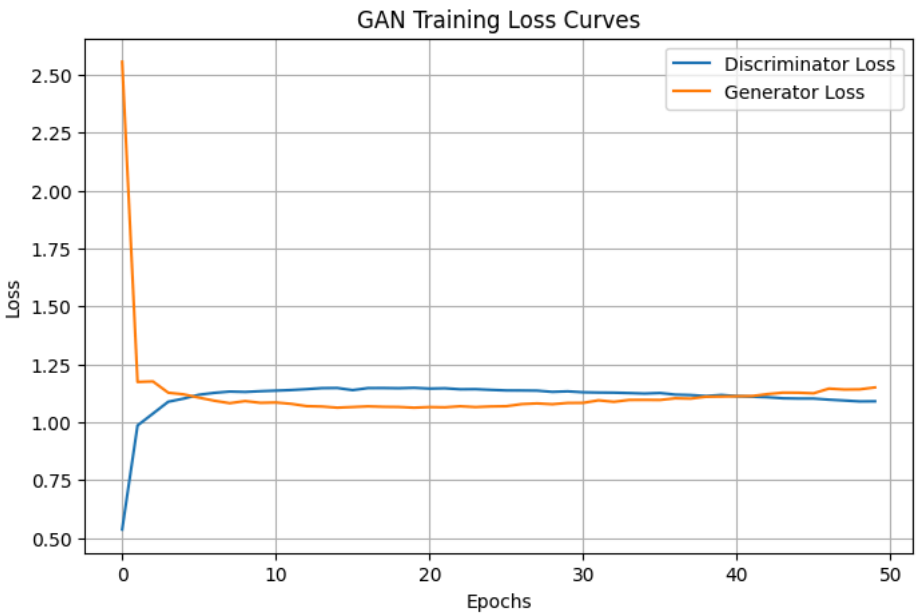
```
plt.plot(d_losses, label="Discriminator Loss")
plt.plot(g_losses, label="Generator Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("GAN Training Loss Curves")
plt.legend()
plt.grid(True)
plt.savefig("gan_loss_curve.png")
plt.show()
```

```
import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models

dataset_choice = 'fashion'       # 'fashion'
epochs = 50
batch_size = 128
noise_dim = 100
learning_rate = 0.0002
save_interval = 5


print("GPU Available:", tf.config.list_physical_devices('GPU'))

os.makedirs("generated_samples_fashion", exist_ok=True)
os.makedirs("final_generated_images_fashion", exist_ok=True)

if dataset_choice == 'mnist':
    (x_train, y_train), _ = tf.keras.datasets.mnist.load_data()
else:
    (x_train, y_train), _ = tf.keras.datasets.fashion_mnist.load_data()

x_train = (x_train.astype("float32") - 127.5) / 127.5
x_train = np.expand_dims(x_train, axis=-1)

BUFFER_SIZE = x_train.shape[0]
dataset = (
    tf.data.Dataset.from_tensor_slices(x_train)
    .shuffle(BUFFER_SIZE)
    .batch(batch_size, drop_remainder=True)
    .prefetch(tf.data.AUTOTUNE)
)

img_shape = (28, 28, 1)

def build_generator():
    return models.Sequential([
        layers.Dense(256, input_shape=(noise_dim,)),
        layers.LeakyReLU(0.2),
        layers.BatchNormalization(),

        layers.Dense(512),
        layers.LeakyReLU(0.2),
```

```
        layers.BatchNormalization(),

        layers.Dense(np.prod(img_shape), activation='tanh'),
        layers.Reshape(img_shape)
    ])

def build_discriminator():
    return models.Sequential([
        layers.Flatten(input_shape=img_shape),
        layers.Dense(512),
        layers.LeakyReLU(0.2),

        layers.Dense(256),
        layers.LeakyReLU(0.2),

        layers.Dense(1, activation='sigmoid')
    ])

generator = build_generator()
discriminator = build_discriminator()

cross_entropy = tf.keras.losses.BinaryCrossentropy()

g_optimizer = tf.keras.optimizers.Adam(learning_rate, 0.5)
d_optimizer = tf.keras.optimizers.Adam(learning_rate, 0.5)

@tf.function
def train_step(real_images):
    batch = tf.shape(real_images)[0]
    noise = tf.random.normal([batch, noise_dim])

    with tf.GradientTape() as disc_tape:
        fake_images = generator(noise, training=True)

        real_output = discriminator(real_images, training=True)
        fake_output = discriminator(fake_images, training=True)

        d_loss_real = cross_entropy(tf.ones_like(real_output), real_output)
        d_loss_fake = cross_entropy(tf.zeros_like(fake_output), fake_output)
        d_loss = d_loss_real + d_loss_fake

    grads_d = disc_tape.gradient(d_loss, discriminator.trainable_variables)
    d_optimizer.apply_gradients(zip(grads_d, discriminator.trainable_variables))

    noise = tf.random.normal([batch, noise_dim])

    with tf.GradientTape() as gen_tape:
        fake_images = generator(noise, training=True)
        fake_output = discriminator(fake_images, training=True)
        g_loss = cross_entropy(tf.ones_like(fake_output), fake_output)

    grads_g = gen_tape.gradient(g_loss, generator.trainable_variables)
    g_optimizer.apply_gradients(zip(grads_g, generator.trainable_variables))

    return d_loss, g_loss

def save_image_grid(epoch, n=5):
    noise = tf.random.normal([n*n, noise_dim])
    gen_imgs = generator(noise, training=False)
    gen_imgs = (gen_imgs + 1) / 2.0

    fig, axs = plt.subplots(n, n, figsize=(5, 5))
    idx = 0
    for i in range(n):
        for j in range(n):
            axs[i, j].imshow(gen_imgs[idx, :, :, 0], cmap='gray')
            axs[i, j].axis('off')
            idx += 1
    fig.savefig(f"generated_samples_fashion/epoch_{epoch:02d}.png")
    plt.close()

d_losses = []
g_losses = []

for epoch in range(1, epochs + 1):
    start = tf.timestamp()

    epoch_d_loss = []
    epoch_g_loss = []

    for real_batch in dataset:
        d_loss, g_loss = train_step(real_batch)
```

```
        epoch_d_loss.append(d_loss.numpy())
        epoch_g_loss.append(g_loss.numpy())

    d_losses.append(np.mean(epoch_d_loss))
    g_losses.append(np.mean(epoch_g_loss))

    elapsed = tf.timestamp() - start

    print(
        f"Epoch {epoch}/{epochs} | "
        f"D_loss: {d_losses[-1]:.3f} | "
        f"G_loss: {g_losses[-1]:.3f} | "
        f"Time: {elapsed:.2f}s"
    )

    if epoch % save_interval == 0:
        save_image_grid(epoch)

noise = tf.random.normal([100, noise_dim])
final_imgs = generator(noise, training=False)
final_imgs = (final_imgs + 1) / 2.0

for i in range(100):
    plt.imshow(final_imgs[i, :, :, 0], cmap='gray')
    plt.axis('off')
    plt.savefig(f"final_generated_images_fashion/img_{i}.png")
    plt.close()

classifier = models.Sequential([
    layers.Flatten(input_shape=img_shape),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

classifier.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

classifier.fit(x_train, y_train, epochs=5, batch_size=128, verbose=0)

predictions = classifier.predict(final_imgs)
predicted_labels = np.argmax(predictions, axis=1)

print("\nLabel Distribution of Generated Images:")
unique, counts = np.unique(predicted_labels, return_counts=True)
for u, c in zip(unique, counts):
    print(f"Class {u}: {c} images")
```

```
Epoch 8/50 | D_loss: 1.132 | G_loss: 1.082 | Time: 3.16s
Epoch 9/50 | D_loss: 1.131 | G_loss: 1.091 | Time: 2.51s
Epoch 10/50 | D_loss: 1.134 | G_loss: 1.084 | Time: 2.54s
Epoch 11/50 | D_loss: 1.136 | G_loss: 1.085 | Time: 2.48s
Epoch 12/50 | D_loss: 1.139 | G_loss: 1.079 | Time: 5.10s
Epoch 13/50 | D_loss: 1.143 | G_loss: 1.069 | Time: 2.57s
Epoch 14/50 | D_loss: 1.147 | G_loss: 1.068 | Time: 2.47s
Epoch 15/50 | D_loss: 1.148 | G_loss: 1.063 | Time: 2.47s
Epoch 16/50 | D_loss: 1.139 | G_loss: 1.066 | Time: 3.12s
Epoch 17/50 | D_loss: 1.147 | G_loss: 1.069 | Time: 2.51s
Epoch 18/50 | D_loss: 1.147 | G_loss: 1.067 | Time: 2.47s
Epoch 19/50 | D_loss: 1.147 | G_loss: 1.066 | Time: 3.36s
Epoch 20/50 | D_loss: 1.148 | G_loss: 1.063 | Time: 3.04s
Epoch 21/50 | D_loss: 1.145 | G_loss: 1.065 | Time: 2.51s
Epoch 22/50 | D_loss: 1.147 | G_loss: 1.064 | Time: 2.87s
Epoch 23/50 | D_loss: 1.142 | G_loss: 1.069 | Time: 2.57s
Epoch 24/50 | D_loss: 1.143 | G_loss: 1.065 | Time: 2.71s
```

```
Epoch 41/50 | D_loss: 1.113 | G_loss: 1.112 | Time: 2.53s
Epoch 42/50 | D_loss: 1.110 | G_loss: 1.113 | Time: 3.20s
Epoch 43/50 | D_loss: 1.108 | G_loss: 1.121 | Time: 2.54s
Epoch 44/50 | D_loss: 1.103 | G_loss: 1.128 | Time: 2.53s
Epoch 45/50 | D_loss: 1.102 | G_loss: 1.127 | Time: 2.56s
Epoch 46/50 | D_loss: 1.102 | G_loss: 1.125 | Time: 3.53s
Epoch 47/50 | D_loss: 1.097 | G_loss: 1.145 | Time: 2.52s
Epoch 48/50 | D_loss: 1.094 | G_loss: 1.141 | Time: 2.48s
Epoch 49/50 | D_loss: 1.090 | G_loss: 1.142 | Time: 2.50s
Epoch 50/50 | D_loss: 1.090 | G_loss: 1.150 | Time: 2.76s
WARNING:tensorflow:6 out of the last 3613 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step

Label Distribution of Generated Images:
Class 0: 9 images
Class 1: 4 images
Class 2: 8 images
Class 3: 7 images
Class 4: 17 images
Class 5: 3 images
Class 6: 22 images
Class 7: 1 images
Class 8: 12 images
```

```
plt.figure(figsize=(8, 5))
plt.plot(d_losses, label="Discriminator Loss")
plt.plot(g_losses, label="Generator Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("GAN Training Loss Curves")
plt.legend()
plt.grid(True)
plt.savefig("gan_loss_curve_fashion.png")
plt.show()
```